Zero-knowledge proof

[TaSK]

juni 2026

1 Innledning

Dette tekniske notatet gir en oversikt over en publisert artikkel [4, 5]. Fordi dette notatet beskriver en artikkel er her ingen nye resultater. Termer som "artikkelen", "kapittel 2", "figur 2", og tilsvarende refererer til [5]. Der det er nødvendig å referere til denne teksten bruker vi eksplisitt "i dette notatet".

Det gjøres ingen antagelser om kanaler som meldinger sendes på, utover ordning i tid (en melding ankommer ikke føær den er sendt). I særdeleshet antas det ikke at kanaler tilbyr konfidensialitet.

2 Bakgrunn

Tema er hvordan Alice kan overbevise Bob at hun kjenner en hemmelighet, uten å hverken fortelle noe av verdi til Bob (ut over at hun kjenner hemmeligheten), og heller ikke til Charlie (som ikke kjenner hemmeligheten) som lytter på samtalen. Det er en fordel om Charlie heller ikke kan konkludere at Alice kjenner hemmeligheten ut fra kommunikasjonen.

For eksempel: Gitt at Alice har konfidensielle data, for eksempel et dokument; la oss kalle dokumentet for D. Hun tilbyr ett eller annet til andre som også har dokumentet, for eksempel en form for login. Bob mener han har det, og ønkser å bevise det til Alice. Når følgende protokoll er ferdig er A overbevist om at B også har (tilgang til) D:

$A \longrightarrow B : N'_A$

$B \longrightarrow A : H\{N'_A \,.\, D\}$

hvor notasjonen $.$ betyr sammensetting (*concatination*), N'_A "et tilstrekkelig stort tilfeldig tall" (la oss si $\approx 2^{256}$), og $H\{m\}$ et digitalit fingeravtrykk (*hashing*) av m. Vi noterer i margen at denne enkle protokollen har mange svakheter og ikke tilfredsstiller moderne krav; den fyller likevel formålet her.

Fordi han lytter på all kommunikasjon kjenner Charlie nå N'_A og $H\{N'_A \,.\, D\}$. Det vil si at om han har stjålet et dokument D' kan han undersøke om $D' = D$ ved å beregne $H\{N'_A \,.\, D'\}$ og sammenligne med $H\{N'_A \,.\, D\}$. Det vil at protokollen mellom A og B lakk (meta)informasjon om D til Charlie.

Evnen til å kunne verifisere om $D = D'$ er en betydelig fordel som kan gi næring til mange typer angrep. *Zero Knowledge* handler om å overbevise den andre om noe, uten å lekke informasjon. For å illustrere hva som menes, la Alice og Bob sitte på hver sin side av bordet, med en kortstokk mellom seg. Alice trekker et kort, og ser på det, og legger det med baksiden opp; kortet var rødt (hjerter eller ruter). Spørsmålet er: Hvordan kan Alice overbevise Bob om at hun trakk et rødt kort uten å vise ham det eller avsløre noe annet av verdi?

La Alice gå gjennom stokken og plukke ut de 26 sorte kortene (kløver og spar) og legge dem på bordet så Bob kan se dem. Under antagelsen om at kortstokken er "ekte" kan han konkludere uten mulighet for å gjøre feil at Alice trakk et rødt kort.

For at denne protokollen skal fungere, må Bob akseptere en rekke svært heftige antagelser. Den første handler om kortstokken; han må anta den har 52 kort, ikke inneholder en *joker*, har 13 kort av hver (kort)farge, samt den fysiske omgivelsen som må etableres (hva innbærer det å sitte på andre siden av bordet?).

Som vi skal se må det alltid gjøres antagelser; styrken i systemet avhenger av hva som legges til grunn. Kompleksitetsteori er verktøyet for å snakke om antagelser.

3 Matematisk utgangspunkt

Den matematiske konteksten i artikkelen er språk og kompleksitetsklasser; vi gir derfor en kort oppfriskning.

3.1 Språk

Alan Turing var den første til å representere (matematiske) spørsmål med symboler (tekst) [7] men det var først 25 år senere at det ble forstått at dersom et problem representeres som en kompakt streng (sekvens av symboler) så vil lengden av strengen være et mål på "størrelsen" av problemet [6]. For en introduksjon til anbefaler vi [2]; første kapittel går nøye gjennom representasjon av problemer.

I spørsmål om involverer tall er det åpenbart hvordan problemet skal representeres. For eksempel: Om spørsmålet er: "Er tallet et primtall?" så er problemet som skal besvares et tall. Tallet (problemet) kan være 323 (som vil si $(101000011)_2$). Vi har altså et spørsmål og et problem (i konktekst av spørsmålet).

Når spørsmålet omhandler en graf, for eksempel om grafen er forbundet, kan kan vi konvertere grafen til en matrise (hvor $M_{ij} = 0$ om det ikke er en kant mellom nodene i og j). Når en slik matrise skrives ut blir resultatet igjen en streng (av 0 og 1). Lengden på dette problemet er naturligvis produktet av dimensjonene.

Når vi bruker strenger til å representere problemer på denne måten kan vi betrakte strengene som et "ord"; hver streng representerer ett (og bare ett) problem. Indirekte har vi nå også fått et mål på størrelse: Jo lenger streng jo større problem. Legg merke til at tallet 1234567890 er ti ganger så stort som tallet 123456789. Men som problem er 1234567890 bare 10% større (lengere).

Problemene til et gitt spørsmål er et (uendelig) sett av strenger. Spørsmålet deler dette settet to: De problemer som har svaret "ja" og de som ikke har svaret "ja". Kombinasjonen av et spørsmål, en representasjon av problemene som skal besvares, settet av alle problemer (ordene), og medlemmene av settet som har svaret "ja", det kaller vi *et språk*.

Språket som inkluderer spørsmålet om X er prim når $X \in \mathbf{N}$ og settet med tall hvor svaret er "ja", det heter PRIMES. Akkurat som ikke alle bokstavkombinasjoner tilhører språket norsk, tilhører ikke alle tall språket PRIMES.

Faktorisering fanges i språket *FACTOR*. Gitt to tall n og k, finnes det et tall d, mindre eller lik k som dividerer n. Beviset er naturligvis d.

CLIQUE er spørsmålet om en graf inneholder en komplett sub-graf (klikk) med en gitt størrslse, sammen med problemene som representerer grafer hvor svaret er "ja".

$$L_{\text{CLIQUE}} = \{\langle G, k\rangle \mid G \text{ har en klikk med størrelse } k\} \tag{1}$$

Her er G en streng som representerer en graf og k et tall (størrelsen på klikken).

3.2 Kompleksitesklasser

Det viser seg at språkene faller naturlig ned i et sett med klasser; vi kaller dem kompleksitetsklasser. Klassene skiller seg fra hverandre ved noen sentrale egenskaper ved de språkene som er med i klassen. Det viser seg også at det er "lett" å konvertere et språk i en klasse til et annet i samme klasse. At noe er "lett" betyr her at det er i klassen **P** (se nedenfor).

Det essensielle er at alle spørsmål i et språk er "like". For eksempel er faktorisering av store tall som er produktet av kun to primfaktorer lik for alle tall av lik størrelse. Det vil si at vi kan bygge en løsning rundt faktorisering uten på forhånd velge hvilke tall som skal inngå. Det gjør at hver instans er "frisk" og med samme egenskaper som alle andre.

3.2.1 Vekst

Når vi skal addere to tall, da legger vi først sammen siste siffer i begge tallene, deretter nest siste, og så videre. Det vil si at kompleksiteten vokser lineært med lengden på tallene (dobbelt så lange tall krever dobbelt så mange operasjoner). Vi ser utelukkende på tallenes lengde, ikke deres "størrelse".

Kompleksiteten (arbeidsmengden) i addisjon vokser lineært med lengden på tallene som skal legges sammen.

Multiplikasjon krever at vi ganger hvert siffer i tallene med de andre, og summerer resultatene til slutt. Mest arbeid er det når tallene er like lange. Kompleksiteten i multiplikasjon er derfor ON^2 hvor N her ikke er tallene men lengden. Addisjonen vi gjør til slutt blir fort neglisjerbar i forhold til selve multiplikasjonen fordi arbeidsmengden vokser mye raskere.

Som regel kan vi bruke tid på å regne noe ut, eller pre-beregne for å tjene tid senere. Det vil si bytte vekst i tid mot vekst i rom. Det viser seg at problemer i en klasse ikke kan "bryte ut" av klassen fordi vekst i rom "koster" like mye som vekst i tid. For eksempel kan vi løse multiplikasjon i fast tid ved å ha en ferdig multiplikasjonstabell. Men den har jo størrelse N^2 og i praksis er derfor ingenting spart. Derfor, når kompleksiteten er, for eksempel, ON^2 er det enten i tid eller i rom.

3.2.2 Klassen P

Et språk det krever $O(N^2)$ beregninger å bestemme er klart lettere å håndtere enn ett som krever $O(N^3)$. Men som alltid er alt relativt: I forhold til $O(2^N)$ er forskjellen mellom $O(N^2)$ og $O(N^3)$ så liten at de er "like".

De språkene som kan konstrueres i tid og rom som ikke vokser mer enn polynomisk relativ til lengden på spørsmålet, de sies å tilhøre klassen P.

For eksempel PARTALL:

$$\textit{lPARTALL} : \{n : n\text{er partall} \tag{2}$$

Algoritmen for å bestemme om et tall er i PARTALL er å se om siste bit er 0 eller 1. Det tar fast tid (uavhengig av lengen).

Gitt en graf G kan vi bruke bredde-først eller dybde-først for å finne om én node er forbundet med en annen.

$$\text{STI} = \{(G, a, b) : \text{Det er en sti fra } a \text{ til } b \text{ i } G \tag{3}$$

Hverken bredde-først eller dybde-først vokser raskene enn antall noder den skal besøke. Det vil si at veksten er polynomisk når størrelsen på G er målet. Språket STI er i **P**:

Det er lett å finne ut om n er prim (prøve å dividere med alle tall $< \sqrt{n}$) om enn tidkrevende om n er stor. Som problem i et språk måler vi N som lengden. Et tall som er dobbelt så *langt* som et annet, for eksempel 20 mot 10 siffer, det er svært mye større. Denne algoritmen gir derfor eksponensiell vekst i antall operasjoner (målt mot N som er lengden av n). Heldigvsi finnes det mer effektive algoritmer; det beste vi kjenner er $\tilde{O}(\log^6 n)$ [1]. Det er avgjørende her å se at å avgjøre om et tall er prim eller ei er noe annet og mye lettere enn å faktisk finne primfaktorene. Språket PRIMES er derfor i **P**.

3.2.3 BPP (Bounded-error Probabilistic Polynomial time)

For at beskrivelsen skal være korrekt nevner vi at fordi det ikke er bevist at *psaudorandomness* i det generelle tilfellet kan erstatte ekte tilfeldighet, så er **P** $\subset$ **BPP**. Men som alle andre antar vi at så er tilfelle, og at **P** $=$ **BPP**.

3.2.4 Klassen NP

En annen klasse, som inkluderer settet **P**, er **NP**. Språk i denne klassen er de hvor det er mulig å verifisere et bevis[1] på at en instans har "ja"-svar i polynomisk tid.

For å se at P er innholdt i NP er det tilstrekkelig å notere at elementene i P kan konstrueres i polynomisk tid. Det vil si det ikke er behov for noe bevis i det hele tatt; man kan bare regne ut løsningen.

Et språk som ofte brukes som eksempel er SAMMENSATT:

$$\text{SAMMENSATT} : \{n : \text{n er sammensatt (ikke prim)}\} \tag{4}$$

[1]På engelsk har *proof* og *evidence* forskjellig betydning. Når vi snakker om *bevis* her mener vi *evidence*.

Et bevis for at n er sammendatt (et "ja"-svar) er $n = xy$. Kompleksiteten ved å verifisere, altså multiplisere, øker med lengden på x og y (som naturligvis øker med lengden på n), men kun polynomisk. Altså er SAMMENSATT i **NP**. Kjernen er at det som skal gjøres er ikke å finne ut om et tall er sammensatt, men undersøke om et bevis er gyldig.

PRIMES er også i **NP** fordi det finnes et bevis som kan verifiseres i polynomisk tid[2]. Dersom n er prim da vil det finnes en $a^{n-1} = 1(= n)$ og for hver primfaktor q av $n-1$ vil $a^{(n-1)/q} \neq 1(= n)$. Et bevis for at n er prim vil være a sammen med faktoriseringen av $n-1$ (og tilsvarende bevis for at de faktorene er prim).

Minner igjen om at å finne ut om n er prim er ett problem mens avgjøre om beviset på at n er prim er gyldig, er et annet. Kravet for at et problem er i **NP** er altså at det finnes et bevis (og ikke nødvendigvis at beviset er "lett" å finne).

3.2.5 Klassen NPC

Noen problemer som har enkel bevis, er svært vanskelig å løse. For eksempel

- TRAVELLING SALESMAN: Gitt en graf med vektede kanter, finnes det en rute med lengde $< k$ som besøker alle noder (byer)? Beviset er ruten, som er lett å verifisere har lengde mindre enn k;
- SEKKEPROBLEMET: Gitt et sett av gjenstander med volum og verdi, finnes det et subsett av gjenstandene som til sammen har verdi $> k$ og som til sammen ikke har større volum en sekken?, og
- FARGELEGGE EN GRAP: Gitt en graf hvor kantene ikke har retning, og et antall farger, finnes det en måte fargelegge nodene slik at ingen node har kant til en annen node med samme farge?

Disse, og mange andre (se [3] for mer enn 300 eksempler) er "like", i betydningen at de har kompleksitet som minst er $O(2^N)$.

Problemene i NPC har vært studert av informatikere og matematikere i flere tiår. Det er ingen tegn på at noen av dem har enkle løsninger.

Definisjon 3.1 (Umulig)**.** Noe er *umulig* dersom det er like vanskelig, eller vanskeligere, enn problemene i NPC.

Denne definisjonen er naturligvis en svært uformell måte å uttrykke hvor mange beregninger som i det generelle tilfelle er nødvendig for å avgjøre om en streng $x \in L$, hvor $L \in$ **NPC**.

Det må understrekes at det er forventningen (les: sannsynligheten) som er hindringen. For eksempel: Gitt et 60 sifret tall tall som er produktet av to 30-sifret primtall, da er det mulig å være heldig å gjette de to faktorene på første forsøk. Sannsynligheten for det er svært liten, og derfor starter vi ikke engang på oppgaven. Noe er derfor *umulig* dersom den forventede arbeidsmengden PER SE gjør at man avstår.

Her noterer vi med en viss interesse at hverken RSA (basert på FACTOR eller ECDSA (basert på DLG) er *umulig* å knekke. Vanskelig, men ikke *umulig*.

4 Problemstilling

På et strukturelt nivå ønsker vi at Charlie, som lytter på kommunikasjonen mellom Alice og Bob, er i samme beregningsmessige posisjon etter å ha lytteat på protokollen mellom Alice og Bob som han var før protokollen startet. Altså at kompleksiteten i å finne x ikke endrer seg gjennom å lytte mens Bob på sin side blir overbevist om at Alice kjenner x.

Vi ønsker å finne et språk L som er slik at Alice kan overbevise Bob om at hun kjenner $x \in L$ uten å formidle noe av verdi om x. Konkret:

[2] PRIMES ble opprinnelig (1975) plassert i **NP**. Senere har det vist seg å være i **P**, altså "enklere" enn man opprinnelig trodde.

- Dersom $x \in L$ da kan Alice overbevise Bob om det med en sannsynlighet $P > 1 - \|x\|^c$ for enhver $c > 0$, og
- Dersom $x \notin L$ da vil Bob kunne avvise at $x \in L$ med en sannsynlighet $P > 1 - \|x\|^c$ for enhver $c > 0$.

Med samme begrunnelsen som vi gav over knyttes det vi ønsker til en sannsynlighet vi kan gjøre så liten vi ønsker (i praksis gjennom å gjennomføre flere runder).

4.1 Isomorfe grafer

Eksempelet med kortstokken vi seksjon **??** illustrerte essensen. For bruk mellom datamaskiner, og i særdeleshet for å formidle hemmeligheter, er det nødvendig med en noe mer komplisert omgivelse.

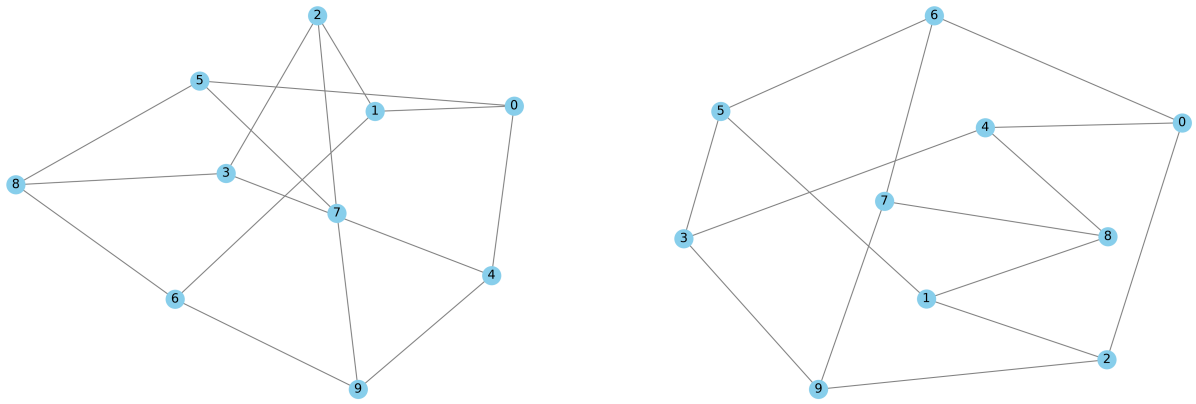
To grafer G_1, G_2 er *isomorfe* dersom de er "like" i sin struktur, men ikke nødvendigvis i navnet på noder og kanter. Formelt er kravet en *bijeksjon* mellom kantene

To grafer $G_1 = (V_1, E_1)$ og $G_2 = (V_2, E_2)$ er *isomorfe* (vi skriver $G_1 \cong G_2$) dersom, og bare dersom, det finnes en *bijeksjon*

$$f : V_1 \to V_2$$

slik at dersom to vilkårlige noder $u, v \in V_1$ er forbundet med en kant i G_1 da er deres avbildinger $f(u)$ og $f(v)$ forbindet med en kant i G_2.

Å generere en graf er lett, og det er enkelt å generere en isomorf kopi av en graf. Det er i praksis "bare" å flytte rundt på det som identifiserer nodene og bygge opp mappingen mens man gir nodene nytt navn. Overgangen mellom to isomorfe grafer G_1 til G_2 er altså en liste som viser hvordan identifikatorene på nodene er knyttet til hverandre. Listen er naturligvis like lang som det er noder i grafen.



(a) Grafen G_1 (b) Grafen G_2

Figure 1: To grafer som er isomorfe. Hele poenget er at det ikke er lett å se. Mappingen her er 0:1, 1:2, 2:0, 3:4, 4:8, 5:5, 6:9, 7:6, 8:3, 9:7.

Selve mappingen er en liste med navn (på noder, se Figur 1. Den forteller ingenting om grafenes struktur (som er det isomorfi handler om). Særlig: Gitt en graf og en mapping kan man (kun) lære at det finnes en isimorf graf, men verdien av den kunnskapen er null.

Det er ikke er lett å avgjøre om to grafer er isomorfe, selv om det ikke er *umulig*. Språket GRAFISOMORFISME er ikke i P, heller ikke i NPC, men sitter i midten (sammen med FACTOR). Gitt en mapping fra én graf til en annen er det trivielt å verifisere at to grafer er isomorfe: GRAFISOMORFE er derfor klart i NP.

Vi skal nå beskrive hvordan Alice kan overbevise Bob om at hun kuenner en hemmelighet, det vil si mappingen mellom to isomorfe grafer, uten å avsløre noe om grafene. Det vil si: en ZERO KNOWLEDGE PROTOCOL.

Grafene G_1 og G_2 er offentlige. Både A og B kjenner mappingen mellom de to grafene, og A ønsker å overbevise B om at så er tilfelle. Protokollen har fire steg somutføres helt til A er fornøyd; vi kommer tilbake til hva det konkret betyr.

$B \longrightarrow A : H$

$B \longrightarrow A : R_\alpha$

$A \longrightarrow B : m$

1. A generer en isomorf kopi H av G_1. Mappingen mellom G_1 og H kaller vi M_1 og mappingen mellom G_2 og H kaller vi M_2. A sender H til B. C ser kun en graf han mistenker er isomorf med G_1 (men kan ikke verifisere det);

2. Etter å ha mottatt H, genererer B et tilfeldig tall $\alpha \in \{1, 2\}$ og sender til A. Vi noterer at dette steget åpner spørsmålet om protokollen er i P eller BPP – et spørsmål vi naturligvis lar ligge;

3. Dersom $\alpha = 1$ da sender A M_1 til B. Dersom $\alpha = 2$ sendes M_2. I begge tilfellene ser C kune en liste med verdier som er like lang som det er noder i grafen. Han lærer ikke noe om grafenes struktur.

4.2 Språkene i artikkelen

Bidraget i artikkelen er to språk QR og QNR som begge har egenskapen at Alice kan overbevise Bob at hun besitter en x som er i språket, uten å avsløre x.

Formelt er **QR** settet av alle heltall (x, y) slik at y er en kvadratisk rest, modulo x. Kvadratisk rest betyr at det finnes et heltall z slik at

$$z^2 \equiv y \pmod{x}$$

, eller med andre ord, at z er kvadratroten til y. Det antas at dersom x er produktet av to primfaktorer, da er det "vanskelig" å finne z gitt y. "Vanskelig" betyr her at det er utenfor **BPP**.

La $x = 77$ (7×11). Da er det langt fra enkelt å avgjøre om $y_1 = 31$ og/eller $y_2 = 37$ er kvadratisk rest i **QR**.[3].

Tilsvarende sier vi at y er *kvadratisk prim* med x dersom en slik z ikke finnes.

Den mest effektive algoritmen er å faktorisere x. Språket FACTOR er ikke *umulig* men man kjenner heller ikke noen løsning i polynomisk tid.

Vi kan nå deinere to språk[4]:

$$QR = (x, y)|y\text{er en kvadratisk rest} \pmod{x} QNR = (x, y)|y\text{er kvadratisk prim} \pmod{x} \quad (5)$$

5 Notasjon

References

[1] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. Primes is in p. *Annals of Mathematics*, 160:781–793, 2004.

[2] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 1st edition, 2009.

[3] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, San Francisco, 1979.

[4] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, STOC '85, pages 291–304. ACM, 1985.

[3] $24^2, 31^2, 46^2$, og $53^2 \equiv 37 \pmod{7}7$ mens 31 er ikke kvadratisk rest $\pmod{7}7$ (det vil si $\nexists z : z^2 \equiv 31 \pmod{7}7$).

[4] Navnene QR og QNR er fra artikkelen.

[5] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[6] Juris Hartmanis and Richard E. Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, 1965.

[7] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1937.